

HOUSEHUNT – HOUSE RENT MANAGEMENT SYSTEM

1. INTRODUCTION

Project Title: HouseHunt – House Rent Management System

Team Members: Individual Project

Introduction:

The **HouseHunt – House Rent Management System** is a modern full-stack web application designed to simplify the process of finding and managing rental houses. In traditional methods, tenants often face difficulties such as lack of verified listings, dependency on brokers, and timeconsuming physical searching. Similarly, house owners struggle to find genuine tenants and manage property listings efficiently.

To solve these challenges, this project provides a centralized digital platform where users can register, log in, and interact with property listings in a structured and efficient manner. The system ensures transparency, reduces manual effort, and improves accessibility for both tenants and property owners.

This application is developed using the **MERN stack (MongoDB, Express.js, React.js, Node.js)**, which is widely used for building scalable and high-performance web applications. The frontend provides an interactive user interface, while the backend manages data processing, authentication, and API handling. MongoDB is used as the database to store user information and property details securely.

The system also includes secure authentication using **JWT (JSON Web Token)** and password encryption using **bcrypt**, ensuring user safety and data protection.

Overall, the project aims to digitize and improve the house rental process by providing a reliable, user-friendly, and efficient platform

2. PROJECT OVERVIEW

PURPOSE OF THE PROJECT:

The primary purpose of the **HouseHunt – House Rent Management System** is to design and develop a modern, efficient, and user-friendly web-based platform that connects house owners and tenants directly without the involvement of intermediaries such as brokers. In traditional rental systems, the process of finding or listing a house is often manual, timeconsuming, and lacks transparency. This project aims to overcome these limitations by providing a centralized digital solution.

In many urban and semi-urban areas, tenants face difficulties in finding suitable rental properties due to scattered information, unreliable listings, and high brokerage charges. Similarly, property owners struggle to find genuine tenants and often depend on brokers, which increases cost and reduces control over the rental process. The HouseHunt system addresses these issues by creating a single platform where both owners and tenants can interact directly in a structured and secure environment.

Core Purpose of the System

The system is developed with the following core intentions:

◆ 1. Eliminating Middlemen

One of the major problems in traditional house renting systems is the dependency on brokers. Brokers often charge high commissions and may provide incomplete or misleading information. This system removes the need for intermediaries by directly connecting property owners and tenants through a digital platform.

◆ 2. Centralized Rental Platform

Instead of searching through multiple sources like newspapers, offline contacts, or scattered websites, HouseHunt provides a **single centralized platform** where all rental listings are available. This improves accessibility and saves time for users.

◆ 3. Improved Transparency

The system ensures transparency by allowing property owners to directly post accurate details such as location, rent, and property images. Users can view complete information before making any decision, reducing confusion and misinformation.

◆ 4. Secure and Authenticated Access

Security is an important aspect of any web application. The HouseHunt system implements **JWT-based authentication** and **bcrypt password encryption**, ensuring that user data is protected and only authorized users can access certain features like property listing management.

◆ 5. Efficient Property Search and Management

The system provides advanced search functionality where users can filter properties based on location, price, and rent. Property owners can easily add, update, or delete their listings, making property management simple and efficient.

◆ 6. Better User Experience

The platform is designed using **React.js**, which provides a responsive and interactive user interface. This ensures that users can access the application smoothly from different devices such as laptops, tablets, and mobile phones.

⚙️ □ Technical Purpose (MERN Perspective)

From a technical perspective, the project also aims to demonstrate the practical implementation of the **MERN stack architecture**, which includes:

- **MongoDB** for flexible and scalable data storage
- **Express.js** for backend routing and API handling
- **React.js** for building dynamic frontend user interfaces
- **Node.js** for server-side execution and logic handling

This combination allows the system to handle real-time requests efficiently and ensures scalability for future enhancements.

Overall Impact

The HouseHunt system ultimately aims to digitalize the house rental process and make it more reliable, transparent, and efficient. By integrating modern web technologies, it reduces manual effort and improves communication between tenants and property owners.

It also serves as a practical implementation of full-stack development concepts, making it a valuable academic project for demonstrating real-world problem-solving using software engineering principles.

Summary

In summary, the purpose of the **HouseHunt – House Rent Management System** is to:

- Simplify house rental processes
 - Remove dependency on brokers
 - Improve transparency and trust
 - Provide a secure and centralized platform
 - Deliver a smooth and user-friendly digital experience
-

FEATURES AND FUNCTIONALITIES

The **HouseHunt – House Rent Management System** includes a wide range of features designed to make the house renting process simple, fast, and efficient. These features are implemented using the **MERN stack architecture (MongoDB, Express.js, React.js, Node.js)**, ensuring high performance and scalability.

The system focuses on providing a smooth experience for both **tenants (users looking for houses)** and **house owners (users posting rental properties)**.

1. User Registration System

The system allows new users to create an account by providing basic details such as name, email, and password.

- Users must register before accessing full features
- Data is securely stored in MongoDB
- Passwords are encrypted using **bcrypt**

- Ensures that only genuine users access the platform

This feature forms the foundation of the authentication system.

◆ 2. Secure Login and Authentication

The login system is implemented using **JWT (JSON Web Token)** authentication.

- Users log in using email and password
- System verifies credentials from the database
- A secure token (JWT) is generated upon successful login
- Token is used to access protected routes

This ensures:

- Security of user accounts
- Prevention of unauthorized access
- Session-based authentication handling

◆ 3. Property Listing System

House owners can list their properties on the platform by providing details such as:

- Location
- Rent amount
- Property price
- Images of the house

Each listing is stored in the MongoDB database and can be viewed by all users.

This feature helps owners advertise their properties easily without broker involvement.

◆ 4. Property Search and Filtering

Users can search for rental properties based on different criteria such as:

- Location
- Price range
- Rent amount

This feature improves usability by allowing users to quickly find suitable houses without manually browsing all listings.

It reduces time consumption and improves efficiency.

◆ 5. Property Management (CRUD Operations)

Property owners have full control over their listings.

They can:

- Create new listings
- Update existing property details
- Delete outdated listings

This is implemented using **CRUD operations (Create, Read, Update, Delete)** in the backend using Express.js and MongoDB.

◆ 6. Responsive User Interface

The frontend is developed using **React.js**, which ensures:

- Fast rendering of components
- Smooth navigation between pages
- Mobile and desktop responsiveness

The UI is designed to be simple, clean, and user-friendly.

◆ 7. Backend API System

The backend is built using **Node.js and Express.js**, which handles all server-side operations.

Key responsibilities include:

- Handling API requests (GET, POST, PUT, DELETE)
- Connecting frontend with database
- Processing authentication logic
- Managing property data

RESTful APIs ensure smooth communication between frontend and backend.

◆ 8. MongoDB Database Integration

MongoDB is used as the primary database due to its flexibility and scalability.

It stores:

- User information
- Property listings
- Authentication data

Data is stored in **JSON-like documents**, making it easy to manage and scale.

◆ 9. Image Upload Support

The system supports uploading property images.

- Helps users visually verify properties
- Increases trust and transparency
- Improves listing quality

(Usually handled using Multer or image URL storage)

◆ 10. Real-Time Data Updates

Any changes made by users (like adding or deleting properties) are reflected immediately in the system without refreshing the entire application.

This improves:

- User experience
- System responsiveness

✦ Summary

The HouseHunt system provides a complete set of features that support both tenants and property owners. It ensures:

- Secure authentication
- Easy property management
- Efficient search functionality

- Responsive UI design
- Reliable backend communication

3. INTRODUCTION TO SYSTEM ARCHITECTURE

The system architecture of the **HouseHunt – House Rent Management System** represents the structured design of how different components of the application interact with each other to deliver the required functionality. It defines the overall framework of the system, including how data flows between the user interface, server-side logic, and database storage.

In modern web development, system architecture plays a crucial role in determining the performance, scalability, security, and maintainability of an application. A well-designed architecture ensures that the system can handle multiple users efficiently, process requests quickly, and manage data securely without conflicts or failures.

The HouseHunt project is built using the **MERN stack architecture**, which is a popular fullstack JavaScript-based architecture consisting of MongoDB, Express.js, React.js, and Node.js. This architecture follows a **three-tier structure**, which separates the application into three main layers: the presentation layer, application layer, and data layer. Each layer has its own responsibility and works independently while communicating with the other layers through welldefined interfaces such as APIs.

Importance of System Architecture in This Project

System architecture is one of the most important parts of this project because it directly impacts how efficiently the HouseHunt application performs. Since this system involves multiple operations such as user authentication, property listing, searching, and data retrieval, a structured architecture ensures that all these processes run smoothly without delay or data inconsistency.

In this project, the architecture ensures:

- Proper separation between frontend, backend, and database
- Smooth communication between client and server using APIs
- Secure handling of user data and authentication
- Efficient storage and retrieval of property information
- Scalability for future enhancements like chat systems or payment integration

□ Design Approach Used

The HouseHunt system follows a **client-server model** combined with a **RESTful API-based architecture**. This means that the frontend (client) is completely separated from the backend (server), and communication happens through HTTP requests.

- The **frontend (React.js)** is responsible for displaying the user interface and handling user interactions.
- The **backend (Node.js + Express.js)** processes the business logic and handles requests.
 - The **database (MongoDB)** stores all persistent data in a structured format.

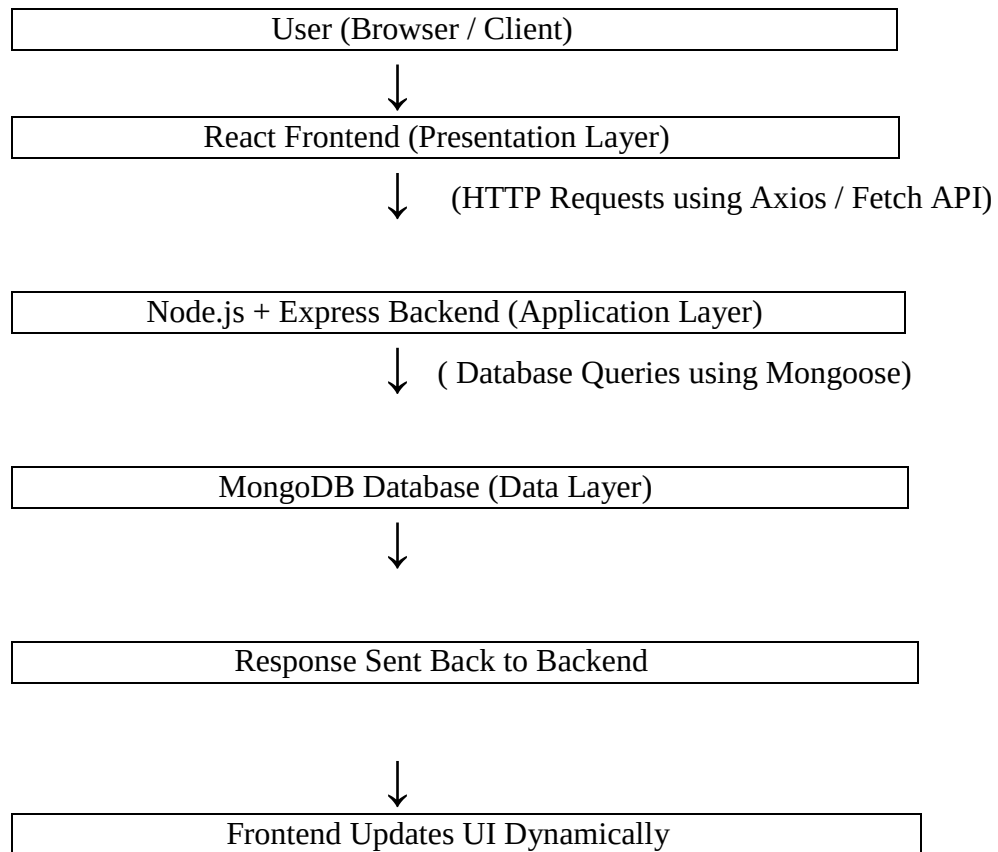
This separation ensures that each component can be developed, tested, and maintained independently.

Data Flow in the Architecture

The flow of data in the HouseHunt system follows a structured process:

1. The user interacts with the application through the web browser.
2. The React frontend captures the user request (for example, login or search property).
3. The frontend sends an HTTP request to the backend server using APIs.
4. The backend receives the request and processes it using Express.js routes.
5. If required, the backend communicates with MongoDB to fetch or store data.
6. MongoDB returns the requested data to the backend.
7. The backend processes the response and sends it back to the frontend.
8. The frontend updates the user interface dynamically based on the response.

🔄 System Architecture Flow



DETAILED EXPLANATION OF COMPONENTS

1 FRONTEND LAYER (REACT.JS)

The frontend layer is responsible for the **visual representation and user interaction** of the system. It is built using **React.js**, a powerful JavaScript library for building dynamic user interfaces.

🔗 Responsibilities of Frontend:

- Displaying login and registration pages
- Showing available property listings
- Handling user inputs through forms
- Sending requests to backend APIs

- Rendering dynamic data from server responses
- Managing navigation between pages

⚙️ Technical Features:

- Uses **component-based architecture**
- Implements **Virtual DOM** for faster updates
- Supports **state management** for dynamic data handling
- Provides **responsive design** for mobile and desktop users

✈️ Advantages:

- Fast rendering of UI components
- Reusable code structure
- Smooth user experience
- Easy integration with backend APIs

◆ 2 BACKEND LAYER (NODE.JS + EXPRESS.JS)

The backend layer is the **core processing unit** of the system. It handles all business logic, authentication, and communication between frontend and database.

🔗 Responsibilities of Backend:

- Handling API requests and responses
- Managing user authentication using **JWT**
- Encrypting passwords using **bcrypt**
- Performing CRUD operations on property data
- Validating user inputs before processing
- Managing server-side logic

⚙️ Technical Features:

- Built using **Node.js runtime environment**
- Uses **Express.js framework** for routing
- Implements **RESTful APIs**
- Middleware-based request handling

✈️ Advantages:

- High performance and scalability
- Lightweight and fast execution
- Easy API management

- Secure data processing

◆ 3 DATABASE LAYER (MONGODB)

MongoDB is used as the database for storing all application data in a flexible, document-based format.

📦 Stored Data Includes:

- User information (name, email, password)
- Property listings (location, price, rent, image)
- Authentication tokens (if required)

⚙️ Technical Features:

- NoSQL document-based database
- Stores data in **JSON-like format**
- Highly scalable and flexible
- Supports fast read/write operations

✈️ Advantages:

- Easy schema modification
- Handles large datasets efficiently
- High performance for real-time applications
- Seamless integration with Node.js

🏠 Example Scenario:

User searches for a house:

- User enters location → React frontend captures input
 - Request sent to backend API `/search`
 - Backend queries MongoDB for matching properties
 - Database returns results
 - Frontend displays filtered properties
-

⚙️ KEY ADVANTAGES OF THIS ARCHITECTURE

The MERN-based architecture provides several benefits that improve system performance and usability:

🔹 1. Separation of Concerns

Each layer (frontend, backend, database) works independently, making development easier.

🔹 2. Scalability

The system can handle increasing users and data without performance issues.

🔹 3. Maintainability

Code is organized into modules, making debugging and updates easier.

🔹 4. Security

Authentication using JWT ensures secure user sessions.

🔹 5. High Performance

Node.js and React.js provide fast execution and rendering.

🔹 6. Reusability

React components can be reused across different parts of the application.

SECURITY IN ARCHITECTURE

Security is a major aspect of the HouseHunt system.

🔒 Security Mechanisms Used:

✓ JWT Authentication

- Ensures secure login sessions
- Generates encrypted tokens for users
- Protects private routes

✓ bcrypt Password Encryption

- Passwords are hashed before storing in database
- Prevents exposure of plain-text passwords
- Enhances data security

✓ Protected API Routes

- Only authenticated users can access certain endpoints
- Prevents unauthorized data access

✓ Input Validation

- Prevents injection attacks
- Ensures clean and valid data entry

★ Importance of Security:

These mechanisms ensure that:

- User data remains safe
- System is protected from hackers
- Only valid users can access services

SUMMARY

The architecture of the **HouseHunt – House Rent Management System** is based on a well-structured **MERN stack three-tier model**. It divides the system into frontend, backend, and database layers, ensuring smooth communication and efficient processing.

This architecture provides:

- High scalability
- Strong security
- Fast performance
- Easy maintenance
- Better user experience

4. SETUP INSTRUCTIONS

INTRODUCTION

This section explains the setup process required to run the **HouseHunt – House Rent Management System** on a local machine. It includes all necessary software installations, project configuration steps, dependency setup, and environment configuration.

The application is built using the **MERN stack (MongoDB, Express.js, React.js, Node.js)**, so proper setup of both frontend and backend environments is required before running the project.

PREREQUISITES

Before running the project, the following software and tools must be installed on the system:

◆ 1. Node.js

Node.js is required to run the backend server and manage packages using npm (Node Package Manager).

✓ Purpose:

- Runs JavaScript on server-side
- Manages backend dependencies
- Executes Express.js server

✓ Installation:

- Download from official website: <https://nodejs.org>
- Install LTS version

- Verify installation using:

```
node -v  
npm -v
```

◆ 2. MongoDB

MongoDB is used as the database to store user and property data.

✓ Purpose:

- Stores application data in JSON-like format
- Handles user and property collections
- Provides fast data retrieval

✓ Installation Options:

- Install MongoDB Compass (GUI tool)
- OR use MongoDB Atlas (cloud database)

✓ Verify Connection:

If using local MongoDB:

```
mongod
```

If using Atlas:

- Create cluster
- Get connection string

◆ 3. Git (Optional but Recommended)

✓ Purpose:

- Clone project from GitHub
- Version control

✓ Check installation:

```
git --version
```

◆ 4. Code Editor (VS Code)

✓ Purpose:

- Write and manage project code
- Run terminal commands
- Manage frontend and backend files

Download: <https://code.visualstudio.com>

◆ 5. Browser (Chrome Recommended)

Used for running and testing React frontend.

PROJECT INSTALLATION STEPS

◆ STEP 1: Clone the Project

First, clone the project repository using Git:

```
git clone https://github.com/your-username/househunt.git
```

Then navigate into the project folder:

```
cd househunt
```

◆ STEP 2: Project Structure Overview

After cloning, the project contains two main folders:

```
househunt/  
├── client/      (Frontend - React)  
└── server/      (Backend - Node.js + Express)
```

◆ STEP 3: Install Backend Dependencies

Navigate to the backend folder:

```
cd server
```

Install required packages:

```
npm install
```

📦 Backend dependencies include:

- express
- mongoose
- bcrypt
- jsonwebtoken (JWT)
- cors
- dotenv

◆ STEP 4: Configure Backend Environment Variables

Create a `.env` file inside the **server** folder:

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_secret_key
```

✓ Explanation:

- PORT → Server running port
- MONGO_URI → MongoDB database connection link
- JWT_SECRET → Secret key for authentication

◆ STEP 5: Start Backend Server

Run the backend using:

```
npm start
```

OR

```
node server.js
```

Backend runs at:

```
http://localhost:5000
```

◆ STEP 6: Install Frontend Dependencies

Open a new terminal and navigate to frontend:

```
cd client
```

Install dependencies:

```
npm install
```

 Frontend dependencies include:

- react
- react-dom
- axios
- react-router-dom

◆ STEP 7: Start Frontend Application

Run React application:

```
npm start
```

Frontend runs at:

```
http://localhost:3000
```

◆ STEP 8: Connect Frontend with Backend

In React frontend, API requests are made using Axios:

Example:

```
axios.get("http://localhost:5001/api/properties")
```

This ensures communication between frontend and backend.

◆ STEP 9: MongoDB Connection Setup

In backend (server.js or db.js):

```
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log("MongoDB Connected"))
  .catch(err => console.log(err));
```

RUNNING THE APPLICATION

After completing setup:

✓ Start Backend:

```
npm start
```

✓ Start Frontend:

```
npm start
```

🌐 Final Output URLs:

- Frontend: <http://localhost:3000>
 - Backend: <http://localhost:5001>
-

TROUBLESHOOTING

● Problem 1: Port already in use

✓ Solution:

```
npx kill-port 5001
```

● Problem 2: MongoDB not connecting

✓ Solution:

- Check MONGO_URI
 - Ensure internet connection (if Atlas used)
-

● Problem 3: Node modules error

✓ Solution:

```
npm install
```

★ SUMMARY

The setup process of the **HouseHunt – House Rent Management System** involves installing Node.js, MongoDB, and required dependencies, configuring environment variables, and running both frontend and backend servers.

5. FOLDER STRUCTURE

Introduction

The HouseHunt – House Rent Management System follows a structured folder organization to separate database models, frontend resources, routing logic, and backend server functionality. The project is organized to improve maintainability, readability, and scalability.

The application consists of frontend pages developed using HTML, CSS, and JavaScript, while the backend is developed using Node.js, Express.js, and MongoDB.

The following structure represents the organization of files and folders used in the project.

Project Folder Structure

```
HouseRent/
├── models/
│   └── houses.js
├── node_modules/
├── public/
│   ├── addhouses.html
│   ├── delete.html
│   ├── houses.html
│   ├── houses.js
│   ├── index.html
│   ├── login.html
│   ├── register.html
│   └── styles.css
├── routes/
├── db.js
├── package.json
├── package-lock.json
└── server.js
```

Description of Main Folders

■ models Folder

The **models** folder contains the database schema definitions used by MongoDB through Mongoose.

`houses.js`

This file defines the structure of property data stored in the database.

The schema contains fields such as:

- Location
- Price
- Rent
- Image

The model acts as a blueprint for storing and retrieving house information from MongoDB.

Purpose of `houses.js`

- Defines house data structure
- Validates property information
- Connects application with MongoDB collections
- Supports Create, Read, Update, and Delete (CRUD) operations

■ node_modules Folder

The **node_modules** folder contains all third-party packages and dependencies required for the project.

These packages are automatically installed using:

```
npm install
```

Examples of installed packages include:

- Express.js
- Mongoose
- bcrypt
- JWT
- CORS

Purpose

- Provides external libraries
- Supports backend functionality
- Manages project dependencies

This folder is automatically generated and generally not modified manually.

■ public Folder

The **public** folder contains all frontend files that are directly accessible through the browser.

This folder represents the user interface of the HouseHunt system.

5.4 Description of Frontend Files

index.html

The home page of the application.

Functions

- Displays welcome content
- Provides navigation links
- Acts as the entry point of the application

register.html

Used for user registration.

Functions

- Accepts user details
- Creates new user accounts
- Sends registration data to backend APIs

Fields

- Name
- Email
- password

login.html

Used for user authentication.

Functions

- Allows users to log into the system
- Verifies credentials
- Grants access to authorized features

houses.html

Displays available house listings.

Functions

- Fetches house data from backend
- Displays property information
- Shows rent and location details

addhouses.html

Allows users to add new property listings.

Functions

- Collect property information
- Submit data to backend
- Store listings in MongoDB

Input Fields

- Location
 - Price
 - Rent
 - Image
-

delete.html

Provides functionality to remove existing property listings.

Functions

- Select property
- Delete property record
- Update database

houses.js

JavaScript file responsible for frontend functionality related to house operations.

Functions

- Fetch property data
- Send API requests
- Update webpage dynamically
- Handle user interactions

styles.css

Contains styling rules for all webpages.

Responsibilities

- Layout design
- Colors and fonts
- Responsive appearance
- User interface consistency

Backend Files

db.js

This file establishes the connection between the application and MongoDB database.

Responsibilities

- Connect to MongoDB
- Handle connection status
- Report connection errors

Importance

Without this file, the application cannot store or retrieve data from the database.

server.js

The main backend file of the application.

Responsibilities

- Start Express server
- Configure middleware
- Define routes
- Process API requests
- Connect frontend and database

Major Operations

- User Registration
- User Login
- House Management

- Database Communication

This file acts as the central controller of the application.

routes Folder

The routes folder is used to organize API endpoints and request handling.

Purpose

- Manage application routes
- Improve code organization
- Separate routing logic from server logic

Although currently minimal, this folder is important for scalability and future development.

Configuration Files

package.json

This file contains project metadata and dependency information.

Includes

- Project name
- Version
- Scripts
- Installed packages

Example scripts:

```
{  
  "start": "node server.js"  
}
```

package-lock.json

This file records the exact versions of installed packages.

Benefits

- Consistent installations
- Dependency management

- Reproducible project setup

Folder Structure Workflow

The workflow of the project can be summarized as:

```
User
↓
HTML Pages (public folder)
↓
JavaScript (houses.js)
↓
Express Server (server.js)
↓
MongoDB Connection (db.js)
↓
House Model (models/houses.js)
↓
MongoDB Database
```

Advantages of the Project Structure

The chosen folder structure provides several advantages:

- Clear separation between frontend and backend
- Easy maintenance and debugging
- Organized database management
- Improved readability
- Faster development process
- Better scalability for future enhancements

Summary

The HouseHunt project follows a structured organization where frontend resources are stored in the **public** folder, database models are maintained in the **models** folder, and backend functionality is handled through **server.js** and **db.js**. This organization ensures efficient development, easier maintenance, and smooth interaction between the user interface, server, and database.

6. RUNNING THE APPLICATION

Introduction

After successfully installing all required software dependencies and configuring the project environment, the next step is to run the application. The HouseHunt – House Rent Management System follows a client-server architecture where the frontend and backend run as separate processes. Both servers must be started correctly to enable communication between the user interface and the database.

The frontend is responsible for displaying the user interface and collecting user input, while the backend processes requests, performs authentication, and interacts with the MongoDB database. Running both servers simultaneously ensures that users can access all application features without interruption.

Overview of Application Startup Process

Before launching the application, ensure that:

- Node.js is installed
- MongoDB is installed and running (or MongoDB Atlas is configured)
- Project dependencies have been installed using npm
- Environment variables are configured properly
- Internet connection is available (if using MongoDB Atlas)

The startup process involves:

1. Starting the MongoDB database
2. Running the backend server
3. Running the frontend application
4. Accessing the application through a web browser

Starting the Frontend Server

The frontend of the HouseHunt application is responsible for rendering web pages and providing an interactive user experience. It communicates with the backend through HTTP requests and dynamically updates the user interface based on the received data.

Steps to Start the Frontend

Step 1: Open Terminal

Open Visual Studio Code and launch the integrated terminal.

Step 2: Navigate to Frontend Directory

Move to the frontend project folder:

```
cd client
```

Step 3: Install Dependencies

If dependencies are not already installed:

```
npm install
```

This command downloads and installs all required frontend packages specified in the package.json file.

Examples of installed packages include:

- React
- React DOM
- Axios
- React Router

Step 4: Start Frontend Server

Run the following command:

```
npm start
```

Step 5: Verify Frontend Execution

After successful startup, the terminal displays messages similar to:

```
Compiled successfully!
```

```
You can now view HouseHunt in the browser.
```

```
Local: http://localhost:5051
```

The browser automatically opens the application.

Frontend URL

```
http://localhost:5051
```

Frontend Startup Workflow

```
User
↓
Open Terminal
↓
Navigate to Client Folder
↓
npm install
↓
npm start
↓
React Development Server Starts
↓
Application Opens in Browser
```

Starting the Backend Server

The backend server handles all business logic, authentication processes, database communication, and API requests.

It is developed using Node.js and Express.js.

Steps to Start the Backend

Step 1: Open a New Terminal

Keep the frontend terminal running and open a second terminal window.

Step 2: Navigate to Backend Directory

Move to the server folder:

```
cd server
```

Step 3: Install Backend Dependencies

If dependencies are not installed:

```
npm install
```

Packages installed include:

- Express
- Mongoose
- JWT

- bcrypt
- dotenv
- CORS

Step 4: Start Backend Server

Execute:

```
npm start
```

Alternatively:

```
node server.js
```

Step 5: Verify Backend Startup

The terminal should display:

```
Server running on port 5000  
MongoDB Connected Successfully
```

This confirms that both the server and database connection are active.

Backend URL

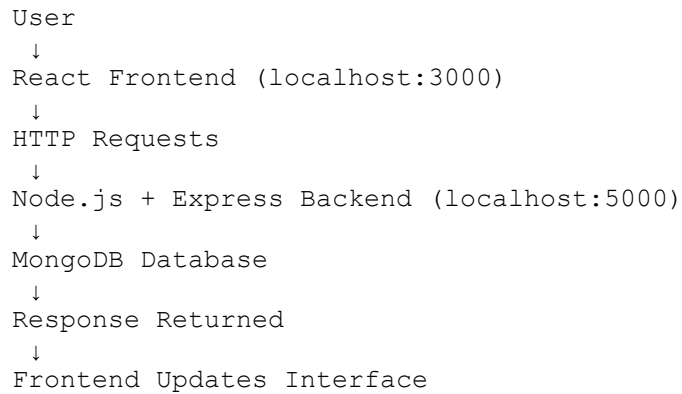
`http://localhost:5000`

Backend Startup Workflow

```
User  
↓  
Open Terminal  
↓  
Navigate to Server Folder  
↓  
npm install  
↓  
npm start  
↓  
Node.js Server Starts  
↓  
Connect to MongoDB  
↓  
API Endpoints Become Available
```

Complete Application Execution Flow

Once both frontend and backend servers are running, the application follows the workflow below:



Example Scenario

1. User enters login credentials.
2. Frontend sends POST request to backend.
3. Backend verifies credentials.
4. MongoDB validates stored user information.
5. Authentication result is returned.
6. Frontend displays login success message.

Running MongoDB Database

If MongoDB is installed locally, start the MongoDB service before running the backend.

Command:

```
mongod
```

Expected output:

```
Waiting for connections on port 27017
```

If MongoDB Atlas is used, ensure that:

- Cluster is active
- Network access is configured

- Connection string is correct in `.env`

Common Errors and Troubleshooting

Error 1: Port Already in Use

Cause

Another application is already using the port.

Solution

```
npx kill-port 3000
```

or

```
npx kill-port 5000
```

Error 2: MongoDB Connection Failed

Cause

Incorrect database connection string.

Solution

- Verify `MONGO_URI` in `.env`
- Check MongoDB service status
- Ensure Atlas cluster is online

Error 3: Module Not Found

Cause

Dependencies are missing.

Solution

```
npm install
```

Error 4: Server Not Starting

Cause

Syntax errors or missing packages.

Solution

Check terminal logs and reinstall dependencies:

```
npm install
```

Best Practices While Running the Application

- Keep frontend and backend running in separate terminals.
- Verify MongoDB connection before testing APIs.
- Store sensitive information in environment variables.
- Monitor terminal logs for runtime errors.
- Test all modules after startup.

Summary

Running the HouseHunt – House Rent Management System involves starting both the frontend and backend environments and ensuring that the MongoDB database is connected successfully. The frontend provides the user interface, while the backend processes requests and communicates with the database. Proper execution of all components ensures smooth functionality of the application and enables users to access rental property services efficiently.

7. API DOCUMENTATION

Introduction

Application Programming Interfaces (APIs) serve as the communication bridge between the frontend and backend of the HouseHunt – House Rent Management System. The backend exposes several RESTful API endpoints that allow users to register, authenticate, manage properties, and retrieve rental information.

The APIs are developed using Node.js and Express.js and communicate with the MongoDB database using Mongoose. These endpoints handle all business logic and ensure secure data exchange between the client and server.

The API layer follows REST architectural principles and uses HTTP methods such as GET, POST, PUT, and DELETE for performing operations.

API Base URL

For local development:

`http://localhost:5001`

All API requests are sent to this server.

Authentication APIs

Authentication APIs are responsible for user registration and login operations.

API 1: User Registration

Endpoint

`POST /register`

Description

Registers a new user into the system.

Request Body

Parameter	Type	Description
name	String	User's full name
email	String	User email address
password	String	User password

Example Request

```
{
  "name": "Madikanti Spandana",
  "email": "spandana@gmail.com",
  "password": "123456"
}
```

Example Response

```
{
  "message": "User registered successfully"
}
```

API 2: User Login

Endpoint

POST /login

Description

Authenticates an existing user and grants access to the system.

Request Body

Parameter	Type	Description
email	String	Registered email
password	String	User password

Example Request

```
{
  "email": "spandana@gmail.com",
  "password": "123456"
}
```

Example Response

```
{
  "message": "Login Successful"
}
```

House Management APIs

These APIs are used for property listing and management.

API 3: Add House

Endpoint

POST /addhouse

Description

Adds a new rental property to the database.

Request Body

Parameter	Type	Description
location	String	House location
price	Number	Property price
rent	Number	Monthly rent
image	String	House image URL

Example Request

```
{
  "location": "Hyderabad",
  "price": 5000000,
  "rent": 15000,
  "image": "house1.jpg"
}
```

Example Response

```
{
  "message": "House added successfully"
}
```

API 4: Get All Houses

Endpoint

GET /houses

Description

Retrieves all available house listings from the database.

Request Parameters

None

Example Response

```
[
  {
    "_id": "12345",
    "location": "Hyderabad",
    "price": 5000000,
    "rent": 15000,
    "image": "house1.jpg"
  },
  {
    "_id": "12346",
    "location": "Warangal",
    "price": 3000000,
    "rent": 10000,
    "image": "house2.jpg"
  }
]
```

API 5: Delete House

Endpoint

DELETE /delete/:id

Description

Deletes a property listing from the database.

URL Parameter

Parameter	Type	Description
id	String	Property ID

Example Request

DELETE /delete/12345

Example Response

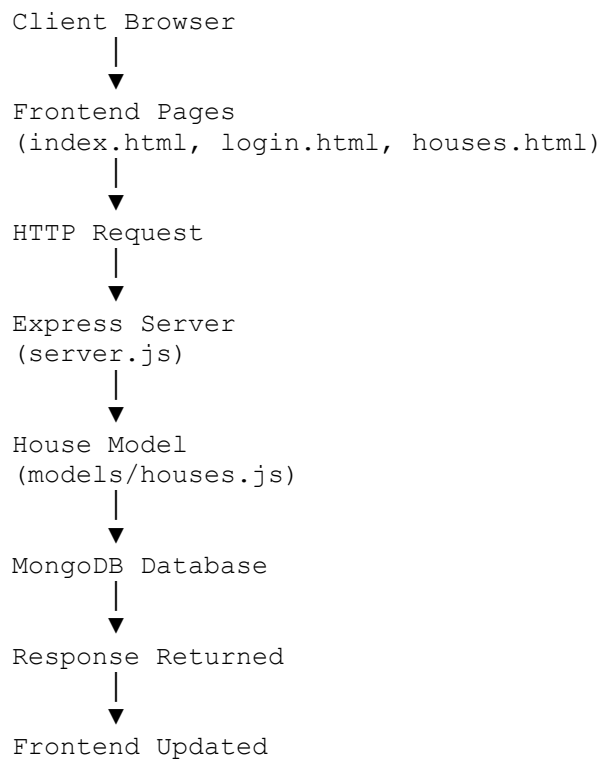
```
{
  "message": "House deleted successfully"
}
```

Database Interaction APIs

The backend interacts with MongoDB through Mongoose models. Every API request follows the sequence:

1. Client sends request.
2. Express route receives request.
3. Controller processes request.
4. Mongoose queries MongoDB.
5. Database returns result.
6. Server sends response.
7. Frontend updates UI.

API Request Flow Diagram



API Security

The HouseHunt backend implements several security mechanisms:

JWT Authentication

- Generates secure authentication tokens.

- Protects restricted routes.
- Validates user identity.

Password Encryption

- Uses bcrypt hashing.
- Prevents storage of plain-text passwords.

Input Validation

- Ensures valid user input.
- Reduces risk of malicious requests.

Error Handling

The backend provides meaningful error messages when requests fail.

Example Error Response

```
{  
  "message": "Invalid credentials"  
}
```

Summary

The API documentation defines all endpoints used by the HouseHunt – House Rent Management System. These APIs enable user authentication, property management, and database interaction. By following RESTful principles and implementing secure authentication mechanisms, the backend ensures reliable communication between the frontend and MongoDB database.

8. AUTHENTICATION AND AUTHORIZATION

Introduction

Authentication and authorization are critical components of any web application. They ensure that only legitimate users can access the system and perform authorized actions. In the HouseHunt – House Rent Management System, authentication is implemented to verify the identity of users, while authorization is used to control access to protected resources and functionalities.

The system utilizes modern security techniques such as password encryption and token-based authentication to safeguard user accounts and sensitive information. These mechanisms help prevent unauthorized access and ensure data security throughout the application.

Authentication Overview

Authentication is the process of verifying the identity of a user before granting access to the application.

In the HouseHunt system, users must register and log in before accessing protected features. During authentication, the system verifies the user's credentials against the information stored in the MongoDB database.

The authentication process consists of:

1. User Registration
2. User Login
3. Credential Verification
4. Token Generation
5. Access to Protected Resources

The implementation ensures that only authenticated users can perform certain operations such as adding or deleting house listings.

User Registration Process

The registration process allows new users to create an account within the system.

Registration Workflow

1. User enters registration details.

2. Frontend sends data to backend API.
3. Backend validates the input.
4. Password is encrypted using bcrypt.
5. User information is stored in MongoDB.
6. Registration success message is returned.

Registration Data

Field	Description
Name	Full name of user
Email	Unique email address
Password	User password

Benefits

- Creates unique user accounts
- Prevents duplicate registrations
- Establishes user identity

Password Security Using bcrypt

Passwords are among the most sensitive pieces of information in any application. Storing passwords in plain text can expose users to significant security risks if the database is compromised.

To address this issue, HouseHunt uses **bcrypt** for password hashing.

What is bcrypt?

bcrypt is a password hashing library that converts plain-text passwords into encrypted hashes before storing them in the database.

Password Storage Process

1. User enters password.
2. Backend generates a salt.
3. bcrypt hashes the password.
4. Encrypted hash is stored in MongoDB.

Example

Original Password:

mypassword123

Stored Hash:

\$2b\$10\$X7A4gQF8W8hP3uQ9V...

Advantages

- Protects user credentials
- Prevents password theft
- Enhances overall system security

User Login Process

After registration, users can access the system through the login page.

Login Workflow

1. User enters email and password.
2. Frontend sends login request.
3. Backend retrieves user information.
4. bcrypt compares entered password with stored hash.
5. If credentials are valid, access is granted.
6. Authentication token is generated.
7. User is redirected to the application.

Login Validation

The system checks:

- Email exists in database
- Password matches stored hash
- User account is valid

Possible Outcomes

Result	Description
Success	User authenticated
Failure	Invalid credentials

Token-Based Authentication

The HouseHunt system uses **JSON Web Tokens (JWT)** for authentication.

JWT is a compact and secure method of transmitting information between parties as a digitally signed token.

Purpose of JWT

- Verify user identity
- Maintain login state
- Protect API endpoints
- Avoid repeated login requests

JWT Structure

A JWT consists of three parts:

Header.Payload.Signature

Example Token

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Token Generation Process

1. User logs in successfully.
2. Backend generates JWT token.
3. Token is sent to frontend.
4. Frontend stores token.
5. Token is attached to future requests.

Authorization Mechanism

Authorization determines what actions a user is allowed to perform after authentication.

While authentication answers:

"Who are you?"

Authorization answers:

"What are you allowed to do?"

In HouseHunt, authorization is used to protect sensitive operations.

Authorized Operations

- Add house listings
- Delete house listings
- Access protected APIs
- Modify property data

Unauthorized Users

Users without valid authentication tokens cannot access protected resources.

Protected Routes

Protected routes are API endpoints that require successful authentication before access is granted.

Examples

```
POST /addhouse
DELETE /delete/:id
```

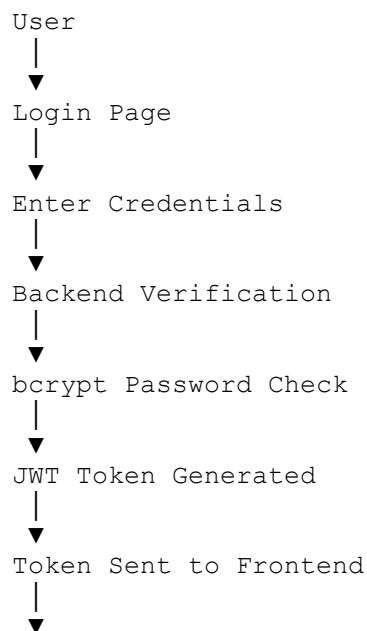
Before executing these requests:

1. Backend checks JWT token.
2. Token validity is verified.
3. User identity is confirmed.
4. Request is processed.

If verification fails:

```
{
  "message": "Unauthorized Access"
}
```

Authentication Flow Diagram



User Authenticated
↓
▼
Access Protected Resources

Session Management

Traditional web applications often use server-side sessions. However, HouseHunt follows a token-based authentication approach.

Session-Based Authentication

Traditional approach:

- Session stored on server
- Session ID stored in browser
- Requires server-side session management

JWT-Based Authentication

HouseHunt uses JWT because:

- No server-side session storage required
- Better scalability
- Faster request processing
- Suitable for MERN stack applications

This approach is often referred to as **stateless authentication**.

Security Benefits

The authentication system provides several security advantages:

Data Protection

- Passwords are encrypted using bcrypt.
- Sensitive information is protected.

Secure Access Control

- Only authenticated users can access protected routes.

Token Verification

- JWT ensures request authenticity.

Reduced Security Risks

- Prevents unauthorized access.
- Minimizes credential exposure.

Error Handling During Authentication

The system handles authentication failures gracefully.

Invalid Email

```
{  
  "message": "User not found"  
}
```

Invalid Password

```
{  
  "message": "Incorrect password"  
}
```

Invalid Token

```
{  
  "message": "Token verification failed"  
}
```

Summary

Authentication and authorization play a crucial role in the HouseHunt – House Rent Management System. The application uses bcrypt for password hashing and JWT for token-based authentication. These mechanisms ensure secure user verification, protect sensitive information, and restrict access to authorized users only. By implementing modern security practices, the system provides a safe and reliable environment for managing rental property information.

9. USER INTERFACE

Introduction

The User Interface (UI) of the HouseHunt – House Rent Management System is designed to provide a simple, user-friendly, and responsive experience for users. The interface enables users to register, log in, browse available properties, add new house listings, and manage existing listings efficiently.

The application uses HTML, CSS, and JavaScript to create an interactive interface that ensures smooth navigation and ease of use.

Home Page Interface

The Home Page serves as the entry point of the application. It provides navigation options for registration, login, and property management.

Features

- Welcome message
- Navigation menu
- Quick access to system modules
- User-friendly design

Figure 9.1: Home Page

 **RentEasy**

House Rent Management System

Register

Login

Browse Homes

Delete Property

Find Your Perfect Home

Enter Location (Hyderabad, Warangal...)

Search Homes

Add Property

House Title

Location

Rent Amount

Add Property

© 2026 RentEasy | House Rent Management System

Description

The home page introduces users to the HouseHunt platform and provides links to important sections of the application.

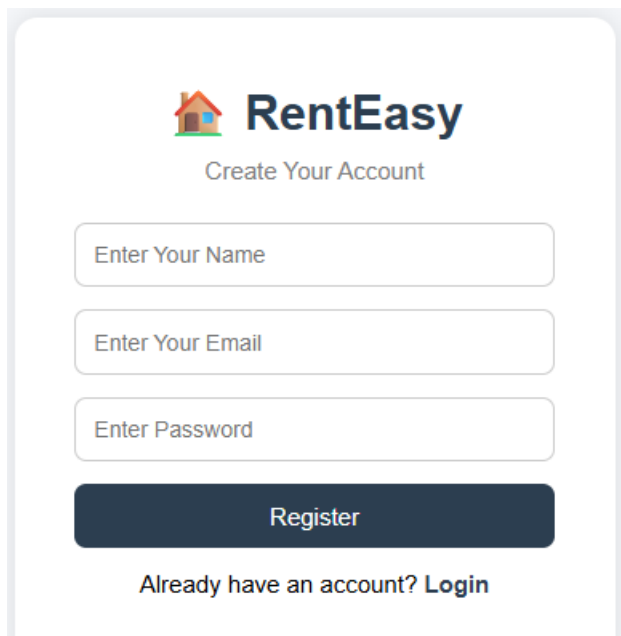
9.3 User Registration Interface

The registration page allows new users to create an account.

Features

- User name field
- Email field
- Password field
- Registration button
- Input validation

Figure 9.2: Registration Page

The image shows a registration form for 'RentEasy'. At the top, there is a logo consisting of a small house icon and the text 'RentEasy'. Below the logo is the text 'Create Your Account'. The form contains three input fields: 'Enter Your Name', 'Enter Your Email', and 'Enter Password'. Below these fields is a dark blue button labeled 'Register'. At the bottom of the form, there is a link that says 'Already have an account? Login'.

Description

Users enter their personal information and submit the registration form. The system validates the information and stores it securely in the database.

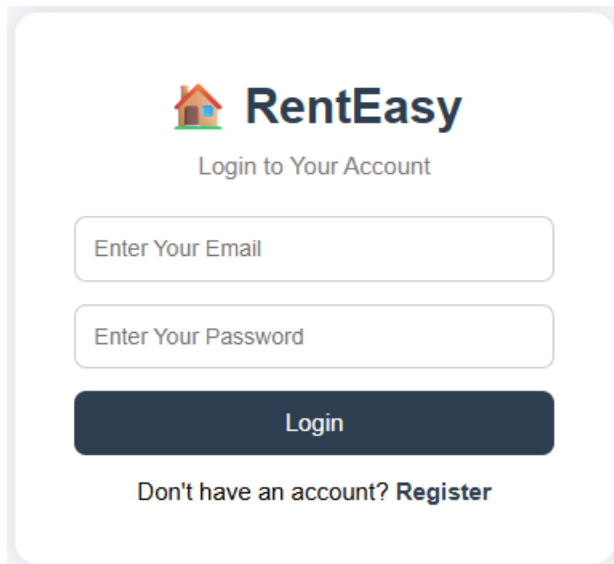
9.4 User Login Interface

The login page authenticates existing users.

Features

- Email input field
- Password input field
- Login button
- Authentication validation

Figure 9.3: Login Page

The image shows a login page for 'RentEasy'. At the top, there is a logo consisting of a small house icon with a red roof and a blue door, followed by the text 'RentEasy' in a bold, dark blue font. Below the logo, the text 'Login to Your Account' is displayed in a smaller, grey font. The main form area contains three elements: a text input field with the placeholder text 'Enter Your Email', another text input field with the placeholder text 'Enter Your Password', and a dark blue button with the text 'Login' in white. At the bottom of the form, there is a link that says 'Don't have an account? Register'.

Description

Registered users enter their credentials to gain access to the system's features.

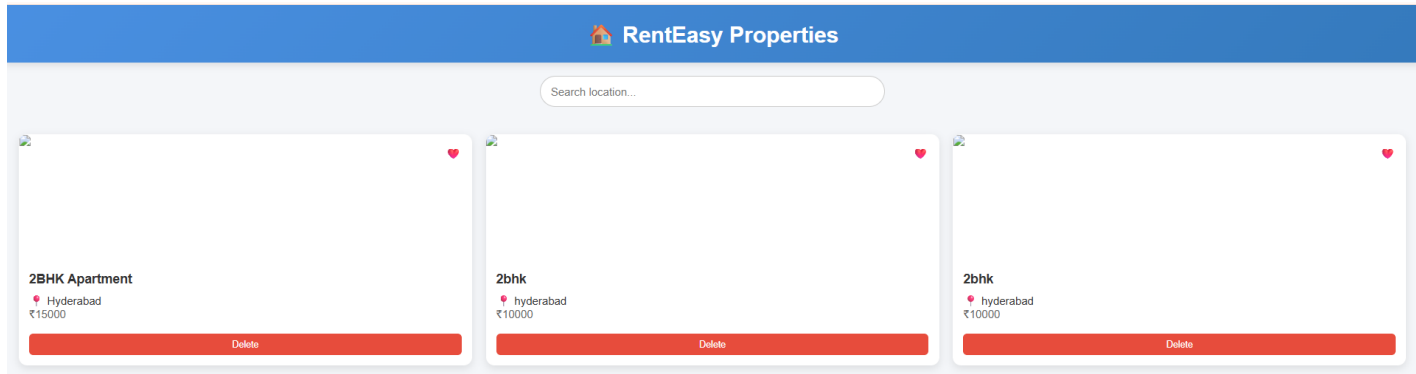
9.5 House Listings Interface

The house listings page displays all available rental properties.

Features

- Property location
- Property price
- Monthly rent
- Property image
- Dynamic data retrieval

Figure 9.4: House Listings Page



Description

The page retrieves property information from MongoDB through backend APIs and displays it in a structured format.

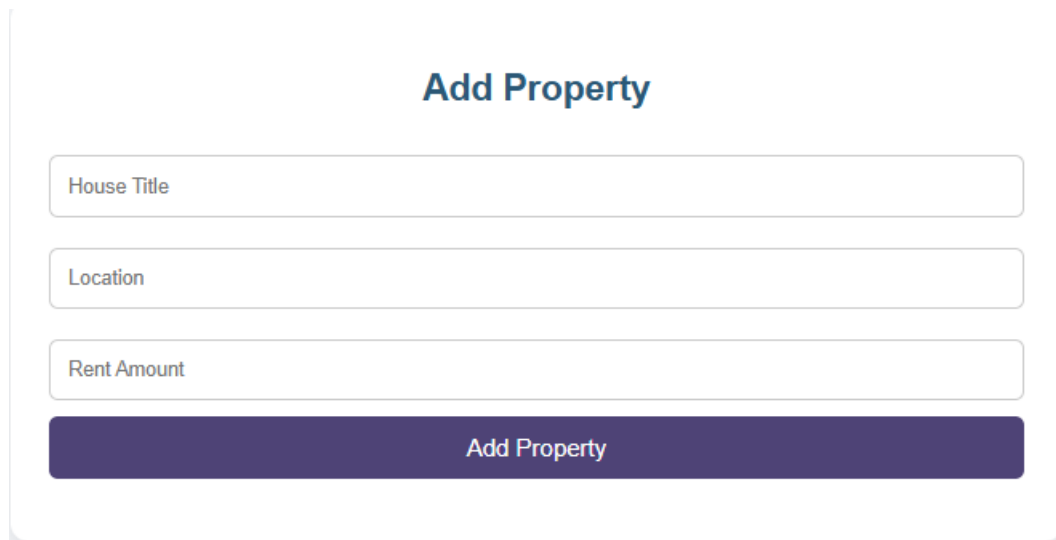
9.6 Add House Interface

The Add House page enables users to create new property listings.

Features

- Location field
- Price field
- Rent field
- Image field
- Submit button

Figure 9.5: Add House Page

A screenshot of a web form titled "Add Property" in a bold, dark blue font. The form is contained within a light gray rounded rectangle. It features three input fields: "House Title", "Location", and "Rent Amount", each with a light gray border and placeholder text. Below these fields is a dark purple button with the text "Add Property" in white. The entire form is set against a light gray background.

Add Property

Description

Users can enter property details and submit them to the database through the backend server.

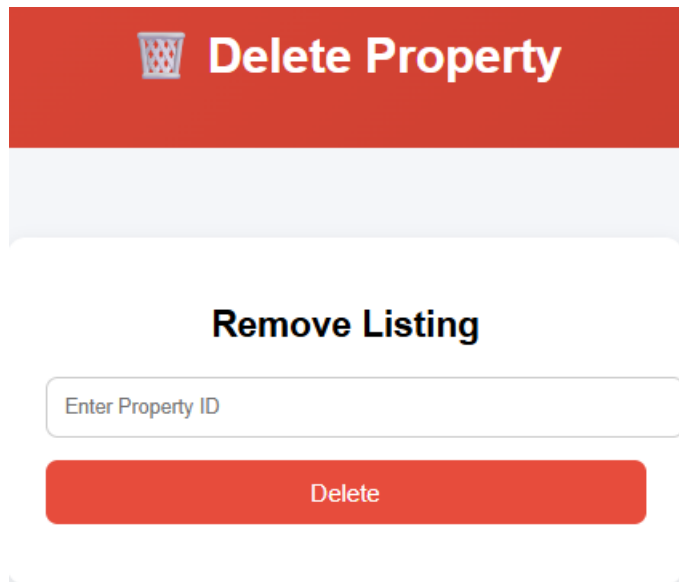
9.7 Delete House Interface

The Delete House page allows removal of existing property records.

Features

- Property selection
- Delete button
- Confirmation message

Figure 9.6: Delete House Page

A UI mockup for a 'Delete Property' feature. It consists of a red header bar with a trash can icon and the text 'Delete Property'. Below this is a light gray box with a white rounded rectangle inside. The rounded rectangle has the title 'Remove Listing' in bold. Under the title is a text input field with the placeholder 'Enter Property ID'. At the bottom of the rounded rectangle is a red button with the text 'Delete' in white.

Delete Property

Remove Listing

Enter Property ID

Delete

Description

This module enables efficient management of property listings by removing outdated or incorrect records.

9.8 User Interface Workflow

```
Home Page
  ↓
Register/Login
  ↓
View House Listings
  ↓
Add New House
  ↓
Delete House
  ↓
Updated Listings Displayed
```

User Interface Design Features

Simplicity

The interface is designed to be easy to understand and operate.

Responsiveness

Pages adapt to different screen sizes and devices.

Consistency

A common color scheme and layout are maintained throughout the application.

Accessibility

Users can easily navigate between modules without technical knowledge.

Summary

The HouseHunt user interface provides a clean and intuitive environment for managing rental properties. Through modules such as Registration, Login, House Listings, Add House, and Delete House, users can efficiently interact with the system. The UI design focuses on simplicity, responsiveness, and usability, ensuring a positive user experience.

10. TESTING

Introduction

Testing is an essential phase in the software development life cycle that ensures the application functions correctly and meets user requirements. The primary objective of testing is to identify defects, verify system functionality, and ensure the reliability of the software before deployment.

In the HouseHunt – House Rent Management System, testing was performed to validate all major functionalities, including user registration, user login, property listing, property addition, property deletion, database connectivity, and API communication. Proper testing helps ensure that the system operates efficiently and provides a seamless user experience.

Objectives of Testing

The main objectives of testing the HouseHunt system are:

- To verify that all modules function according to requirements.
- To identify and fix software defects.
- To validate user inputs and system outputs.
- To ensure smooth communication between frontend, backend, and database.
- To evaluate system performance and reliability.
- To improve overall software quality.

Testing helps reduce errors and increases user confidence in the application.

Testing Strategy

The HouseHunt project follows a systematic testing strategy to verify the correctness of each module.

The testing process consists of:

1. Unit Testing

Individual components and functions are tested independently.

2. Integration Testing

Interaction between frontend, backend, and database is tested.

3. System Testing

The complete application is tested as a whole.

4. Functional Testing

Each feature is tested against specified requirements.

5. User Acceptance Testing

The system is evaluated from the user's perspective.

Testing Tools Used

The following tools were used during testing and development:

Tool	Purpose
Visual Studio Code	Code development and debugging
Google Chrome	Application testing
Node.js	Backend execution
MongoDB Compass	Database testing
Express.js	Backend request handling

These tools helped verify application functionality and identify potential issues.

Unit Testing

Unit testing focuses on testing individual modules separately.

Modules Tested

- Registration Module
- Login Module
- Add House Module
- Delete House Module
- House Listing Module

Example

When testing the registration module:

Input

Name: Spandana
Email: spandana@gmail.com
Password: 123456

Expected Result

User registered successfully

Actual Result

User registered successfully

Status: Pass

Integration Testing

Integration testing verifies communication between different system components.

Components Tested

- Frontend ↔ Backend
- Backend ↔ MongoDB
- Frontend ↔ APIs

Example

User submits a new property listing.

Process:

```
Frontend Form
  ↓
API Request
  ↓
Backend Server
  ↓
MongoDB Database
  ↓
Response Returned
```

Result:

Property information successfully stored and displayed.

Status: Pass

Functional Testing

Functional testing ensures each feature performs according to specifications.

Test Case 1: User Registration

Test Case ID	TC01
Module	Registration
Input	Valid user details
Expected Result	Account created
Actual Result	Account created
Status	Pass

Test Case 2: User Login

Test Case ID	TC02
Module	Login
Input	Valid credentials
Expected Result	Login successful
Actual Result	Login successful
Status	Pass

Test Case 3: Add House

Test Case ID	TC03
Module	Add House
Input	Valid property information
Expected Result	House added
Actual Result	House added
Status	Pass

Test Case 4: View Houses

Test Case ID	TC04
Module	House Listing

Test Case ID	TC04
Input	Request property data
Expected Result	Houses displayed
Actual Result	Houses displayed
Status	Pass

Test Case 5: Delete House

Test Case ID	TC05
Module	Delete House
Input	Valid property ID
Expected Result	House removed
Actual Result	House removed
Status	Pass

Database Testing

Database testing verifies storage and retrieval of information.

Areas Tested

- MongoDB connection
- User data storage
- Property data storage
- Data retrieval operations
- Data deletion operations

Test Results

Operation	Expected Result	Status
Insert User	Success	Pass
Login Validation	Success	Pass
Add Property	Success	Pass
Retrieve Property	Success	Pass
Delete Property	Success	Pass

API Testing

API testing was performed using Postman.

APIs Tested

API Endpoint	Method	Result
/register	POST	Pass
/login	POST	Pass
/addhouse	POST	Pass
/houses	GET	Pass
/delete/:id	DELETE	Pass

Example API Test

Request:

```
POST /login
```

Response:

```
{
  "message": "Login Successful"
}
```

Result: Pass

User Interface Testing

The user interface was tested to ensure:

- Proper page navigation
- Form validation
- Button functionality
- Layout consistency
- Responsive behavior

Pages Tested

- Home Page
- Registration Page
- Login Page
- House Listing Page
- Add House Page
- Delete House Page

All pages functioned correctly during testing.

Error Handling Testing

The application was tested for invalid inputs and unexpected conditions.

Scenario 1: Invalid Login

Input:

Email: wrong@gmail.com
Password: incorrect

Expected Result:

Invalid Credentials

Status: Pass

Scenario 2: Empty Registration Fields

Expected Result:

Validation Error

Status: Pass

Scenario 3: Database Connection Failure

Expected Result:

Connection Error Message

Status: Pass

User Acceptance Testing (UAT)

User Acceptance Testing was conducted to determine whether the system meets user expectations.

Evaluation Criteria

- Ease of use

- Navigation simplicity
- Performance
- Feature completeness
- Reliability

Feedback

Users found the application:

- Easy to navigate
- Simple to use
- Fast in retrieving property information
- Helpful for managing rental listings

Testing Summary Table

Module	Testing Type	Status
Registration	Functional	Pass
Login	Functional	Pass
Add House	Functional	Pass
House Listing	Functional	Pass
Delete House	Functional	Pass
Database	Integration	Pass
APIs	API Testing	Pass
User Interface	UI Testing	Pass

Summary

Testing played a vital role in ensuring the quality and reliability of the HouseHunt – House Rent Management System. Various testing techniques, including unit testing, integration testing, functional testing, database testing, API testing, and user acceptance testing, were conducted. The results confirmed that all major modules operated successfully and met the intended requirements. The application demonstrated stable performance, accurate functionality, and effective interaction between the frontend, backend, and MongoDB database.

11. SCREENSHOTS OR DEMO

Introduction

Screenshots and demonstrations provide visual evidence of the successful implementation of the HouseHunt – House Rent Management System. They help users, evaluators, and project reviewers understand the functionality, user interface, and workflow of the application.

This section presents screenshots of the major modules developed during the project. These screenshots demonstrate the working of the application and validate that all features have been successfully implemented.

Home Page

The Home Page serves as the entry point of the application. It provides navigation links to different modules such as registration, login, house listings, and property management.

Features Demonstrated

- Welcome page
- Navigation menu
- User-friendly interface
- Access to application modules

```

body>
<header>
<h1>🏠 RentEasy</h1>
<p>House Rent Management System</p>

<div class="nav-buttons">
<a href="/register.html">Register</a>
<a href="/login.html">Login</a>
<a href="/houses.html">Browse Homes</a>
<a href="/delete.html">Delete Property</a>
</div>
</header>

<!-- SEARCH SECTION -->
<section class="search-container">
<h2>Find Your Perfect Home</h2>

<input
  type="text"
  id="homeSearch"
  placeholder="Enter Location (Hyderabad, Warangal...)"

  <button onclick="goToListings()">
    Search Homes
  </button>
</section>

<!-- ADD PROPERTY SECTION -->
<section class="form-container">

<h2>Add Property</h2>

<form action="/add-house" method="POST">

  <input
    type="text"
    name="title"
    placeholder="House Title"
    required>

  <input
    type="text"
    name="location"
    placeholder="Location"

```

```

    required>

    <input
      type="number"
      name="rent"
      placeholder="Rent Amount"
      required>

    <button type="submit">
      Add Property
    </button>

  </form>

</section>

<footer>
<p>© 2026 RentEasy | House Rent Management System</p>
</footer>

<script>
function goToListings() {

  let location =
    document.getElementById("homeSearch").value;

  if(location === "") {
    window.location.href = "/houses.html";
  } else {
    window.location.href =
      "/houses.html?search=" + location;
  }

}
</script>

</body>
</html>
...

```



(Ctrl)

Activ
Go to

11.3 User Registration Module

The Registration Page enables new users to create an account.

Features Demonstrated

- User registration form
- Input fields for name, email, and password
- Form submission functionality

Figure 11.2: Registration Page

```
<body>

<div class="container">

  <h1>🏠 RentEasy</h1>
  <p>Create Your Account</p>

  <form action="/register" method="POST">

    <input
      type="text"
      name="name"
      placeholder="Enter Your Name"
      required>

    <input
      type="email"
      name="email"
      placeholder="Enter Your Email"
      required>

    <input
      type="password"
      name="password"
      placeholder="Enter Password"
      required>

    <button type="submit">
      Register
    </button>

  </form>

  <div class="login-link">
    Already have an account?
    <a href="/login.html">Login</a>
  </div>

</div>

</body>
</html>
```

Description

This page allows users to register and store their details securely in the MongoDB database.

11.4 User Login Module

The Login Page authenticates registered users.

Features Demonstrated

- Login form
- Email and password validation

- Authentication functionality

Figure 11.3: Login Page

```
<body>

<div class="container">

  <h1>🏠 RentEasy</h1>
  <p>Login to Your Account</p>

  <form action="/login" method="POST">

    <input
      type="email"
      name="email"
      placeholder="Enter Your Email"
      required>

    <input
      type="password"
      name="password"
      placeholder="Enter Your Password"
      required>

    <button type="submit">
      Login
    </button>

  </form>

  <div class="register-link">
    Don't have an account?
    <a href="/register.html">Register</a>
  </div>

</div>

</body>
</html>
```

Description

Users can enter their credentials and gain access to authorized features of the application.

11.5 House Listing Module

The House Listing Page displays all available rental properties stored in the database.

Features Demonstrated

- Property information display
- Dynamic data retrieval
- Property browsing

Screenshot

Figure 11.4: House Listings Page

```
<body>

<header>
  <h1>🏠 RentEasy Properties</h1>
</header>

<!-- SEARCH BAR -->
<div class="search-box">
  <input type="text" id="search" placeholder="Search location...">
</div>

<!-- CARDS CONTAINER -->
<div id="housesContainer" class="cards"></div>

<!-- JS -->
<script src="houses.js"></script>

</body>
</html>
```

Description

The page fetches property information from MongoDB and displays it in a structured format.

11.7 Delete House Module

The Delete House Page enables users to remove property listings.

Features Demonstrated

- Property selection
- Record deletion
- Database update

Screenshot

Figure 11.6: Delete House Page

```

<body>

<header>
  <h1>🏠 Delete Property</h1>
</header>

<div class="container">
  <h2>Remove Listing</h2>

  <input type="text" id="propertyId" placeholder="Enter Property ID">

  <button onclick="deleteProperty()">Delete</button>

  <p class="msg" id="msg"></p>
</div>

<script>
async function deleteProperty() {
  const id = document.getElementById("propertyId").value;

  if (!id) {
    alert("Please enter Property ID");
    return;
  }

  try {
    const res = await fetch(`/delete/${id}`, {
      method: "DELETE"
    });

    const data = await res.json();

    document.getElementById("msg").innerText = data.message;

    document.getElementById("propertyId").value = "";
  } catch (error) {
    document.getElementById("msg").innerText = "Error deleting property";
  }
}
</script>

</body>
</html>

```

Description

This module allows users to manage existing listings by deleting outdated or unnecessary records.

11.8 MongoDB Database Records

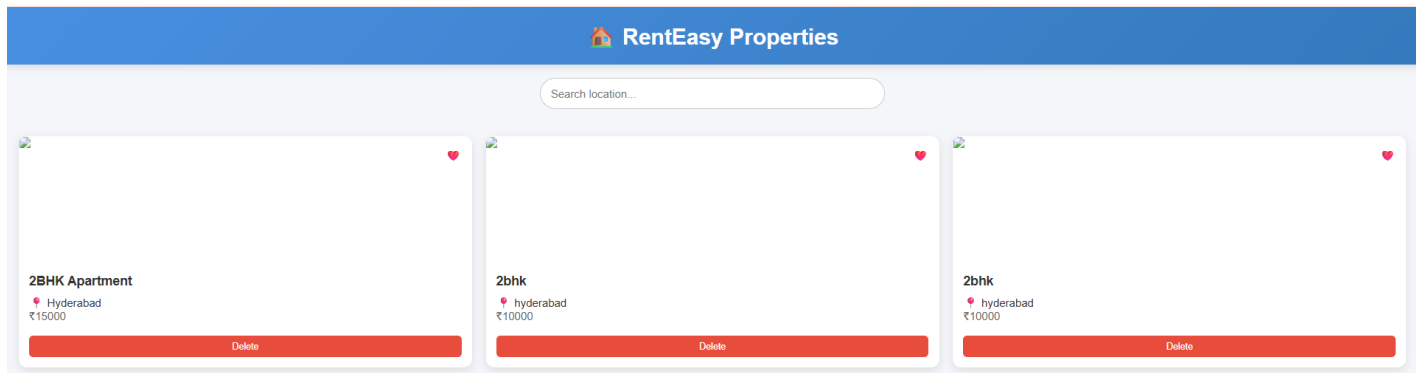
To demonstrate successful database integration, screenshots of MongoDB collections can be included.

Features Demonstrated

- User records

- Property records
- Database connectivity

Figure 11.7: MongoDB Collection View



Description

The screenshot confirms that data entered through the application is successfully stored in MongoDB.

Application Workflow Demonstration

The complete workflow of the HouseHunt system can be represented as:

```

User Opens Application
    ↓
Registration/Login
    ↓
Authentication Successful
    ↓
View House Listings
    ↓
Add New Property
    ↓
Store Data in MongoDB
    ↓
Retrieve Updated Listings
    ↓
Delete Property (Optional)
    ↓
Updated Database Records
  
```

Demo Link

If the project is hosted online, provide the deployment URL below:

[Live Demo](#)

<https://your-demo-link.com>

[GitHub Repository](#)

<https://github.com/spandana/househunt>

If the application is not deployed, mention:

The project was executed and demonstrated locally using Node.js, Express.js, MongoDB, HTML, CSS, and JavaScript. Screenshots of all major modules have been provided as proof of successful implementation.

Summary

The screenshots and demonstrations presented in this section provide visual confirmation of the successful implementation of the HouseHunt – House Rent Management System. They illustrate the functionality of major modules, database integration, and overall application workflow. These screenshots serve as proof that the system operates as intended and fulfills the project requirements.

12. KNOWN ISSUES

Introduction

Despite the successful implementation of the HouseHunt – House Rent Management System, certain limitations and known issues exist within the current version of the application. These issues do not prevent the system from functioning but may affect scalability, usability, security, or user experience. Identifying these limitations helps developers understand areas that require improvement in future versions.

Limited Property Search Functionality

Currently, the application displays available properties and allows users to browse listings. However, advanced filtering options are limited.

Current Limitation

Users cannot filter properties using multiple criteria simultaneously, such as:

- City or area
- Rent range
- Property type

- Number of rooms

Impact

Users may need to manually browse multiple listings to find suitable properties.

Image Handling Limitations

The current implementation stores property image information using image URLs or simple image references.

Current Limitation

- Direct image upload functionality is limited.
- Large image files may not be managed efficiently.
- No image compression mechanism exists.

Impact

Property images may not always display consistently if image links become unavailable.

Basic Authentication Implementation

The application provides user registration and login functionality; however, advanced security features are not fully implemented.

Current Limitation

- Password recovery functionality is unavailable.
- Email verification is not implemented.
- Multi-factor authentication is not supported.

Impact

Account recovery and additional security mechanisms are currently unavailable.

User Role Management

The current version treats all users similarly.

Current Limitation

There is no distinction between:

- Property Owners
- Tenants
- Administrators

Impact

Different levels of access control cannot be enforced.

Limited Error Handling

Although common errors are handled, some unexpected situations may generate generic error messages.

Examples

- Database connection failures
- Invalid API requests
- Network interruptions

Impact

Users may not always receive detailed information regarding the cause of an error.

Mobile Responsiveness

The application is primarily designed and tested on desktop systems.

Current Limitation

Certain layouts may require optimization for:

- Mobile phones
- Tablets
- Small-screen devices

Impact

User experience may vary across different screen sizes.

Deployment Limitation

The current version is primarily executed in a local development environment.

Current Limitation

- No cloud deployment implemented.
- Public hosting not configured.
- Production server not established.

Impact

Users must run the application locally for testing and demonstration purposes.

Performance with Large Data Volumes

The application performs efficiently with small and medium datasets.

Current Limitation

Performance has not been extensively tested with:

- Thousands of users
- Large property databases
- Heavy concurrent traffic

Impact

Additional optimization may be required for large-scale deployment.

Summary

While the HouseHunt system successfully fulfills its primary objectives, certain limitations exist regarding search functionality, image handling, security, user management, responsiveness, and deployment. These issues provide valuable opportunities for future enhancement and system improvement.

13. FUTURE ENHANCEMENTS

Introduction

Future enhancements represent additional features and improvements that can be implemented to increase the functionality, security, performance, and user experience of the HouseHunt – House Rent Management System. These enhancements would help transform the application into a comprehensive and industry-ready rental property platform.

Advanced Property Search and Filters

One of the most important future improvements is the implementation of advanced search functionality.

Proposed Features

- Search by city
- Search by locality
- Price range filtering
- Rent range filtering
- Property type selection
- Number of bedrooms filtering

Benefits

Users can locate suitable properties more quickly and efficiently.

Image Upload and Cloud Storage

Future versions can support direct image uploads.

Proposed Features

- Upload images from local devices
- Cloud storage integration
- Multiple property images
- Image compression and optimization

Benefits

Property listings become more attractive and informative.

User Role Management

Role-based access control can be introduced.

Proposed User Roles

- Administrator
- Property Owner
- Tenant

Benefits

Different users receive appropriate permissions and functionality.

Online Booking and Rental Requests

The application can support direct rental requests.

Proposed Features

- Request property visits
- Booking appointments
- Rental application submission
- Approval workflow

Benefits

Simplifies communication between owners and tenants.

Integrated Chat System

A messaging system can be incorporated.

Proposed Features

- Real-time chat
- Owner-tenant communication
- Notification system

Benefits

Improves interaction and reduces communication delays.

Location Services and Maps

Google Maps integration can be added.

Proposed Features

- Property location display
- Nearby facilities information
- Route navigation

Benefits

Users can better evaluate property locations.

Email and SMS Notifications

Automated notification systems can enhance communication.

Proposed Features

- Registration confirmation emails
- Property inquiry notifications
- Password reset emails
- SMS alerts

Benefits

Keeps users informed about important activities.

Enhanced Security Features

Future versions can implement additional security mechanisms.

Proposed Features

- Email verification

- Password recovery
- Two-factor authentication (2FA)
- Advanced JWT security
- Activity monitoring

Benefits

Improves user trust and data protection.

Mobile Application Development

A dedicated mobile application can be developed.

Proposed Platforms

- Android
- iOS

Benefits

Users can access the platform anytime and anywhere through mobile devices.

Cloud Deployment

The application can be deployed on cloud platforms.

Possible Platforms

- AWS
- Microsoft Azure
- Google Cloud Platform
- Render
- Railway

Benefits

- Global accessibility
 - Improved reliability
 - Better scalability
-

Artificial Intelligence Recommendations

AI-based recommendation systems can be introduced.

Proposed Features

- Personalized property suggestions
- Smart search recommendations
- User preference analysis

Benefits

Improves user experience and property discovery.

Summary

The future scope of the HouseHunt – House Rent Management System is extensive. Enhancements such as advanced search filters, cloud-based image storage, role-based access control, online booking, integrated messaging, map services, cloud deployment, and AI-driven recommendations can significantly improve the application's functionality and usability. These developments would transform HouseHunt into a complete and scalable rental property management platform capable of serving a larger user base and meeting modern industry requirements.